Notes: Conlon and Gortmaker (RAND J Econ, 2020)

Best Practices for Differentiated Products Demand Estimation with PyBLP

Contents

1 Big picture

- **Question.** How should researchers estimate Berry–Levinsohn–Pakes style differentiated-products demand models in a way that is numerically reliable, statistically well behaved, and easy to replicate?
- **Setting.** The paper studies the computational and econometric practice of estimating random-coefficients logit and related BLP-type demand systems. It also introduces and documents `PyBLP`, a Python package for estimating these models.
- **Why this paper matters.** BLP models are central in empirical industrial organization because they are used for merger simulation, valuation of new goods, analysis of product networks, and welfare calculations. But many researchers implement the estimator from scratch, which creates scope for fragile code, inconsistent conventions, and hard-to-replicate results.
- **Core challenge.** The BLP problem has a nonlinear inversion from market shares to mean utilities. The nonlinear parameters enter through this inversion, so estimation requires repeatedly solving inner fixed points and optimizing a nonconvex simulated GMM objective.
- **Main methodological contribution.** The paper collects best practices for:
  - absorbing high-dimensional fixed effects,
  - solving the share inversion,
  - computing analytic gradients,
  - choosing numerical integration rules,
  - solving counterfactual pricing equilibria,
  - constructing feasible approximations to optimal instruments,
  - and checking optimization reliability.
- **Main conceptual contribution.** The paper reformulates the nested fixed-point problem so that simultaneous supply and demand estimation can accommodate high-dimensional fixed effects. It also derives an expression for optimal instruments that makes the supply-side overidentifying restrictions explicit.
- **Main empirical/computational result.** Under good numerical practices, multiple local optima appear to be much less severe than suggested by some prior work. Parameter estimates and counterfactual objects become stable across optimizers and starting values.
- **Main statistical result.** Feasible optimal instruments are extremely useful, especially when combined with correctly specified supply-side restrictions. Supply-side restrictions can substantially improve finite-sample performance when cost shifters are weak.
- **Economic takeaway.** Many apparent BLP estimation problems are not inherent failures of the model. They often come from weak instruments, loose tolerances, poor numerical integration, or unstable fixed-point routines. With careful implementation, BLP estimation is more reliable than the pessimistic view suggests.

2 Data and measurement (key objects)

2.1 Observed objects in a BLP-type problem

- **Product-market panel.** A standard BLP application observes products j in markets t.
- **Core demand-side observables.**
  - observed market shares S_{jt},
  - prices p_{jt},
  - exogenous product characteristics x_{jt},
  - excluded demand shifters v_{jt}, when available,
  - market size, used to map sales into shares.
- **Core supply-side observables.**
  - firm ownership or holdings matrix H_t,
  - cost shifters w_{jt} excluded from demand,
  - product characteristics entering marginal cost,
  - observed prices and shares, which are used to back out markups and marginal costs.
- **Unobservables.**
  - demand shock ξ_{jt},
  - marginal cost shock ω_{jt},
  - consumer-level taste heterogeneity μ_{ijt},
  - simulation error from approximating the distribution of heterogeneity.
- **Key interpretation.** The model treats prices and shares as equilibrium objects. The researcher observes them, inverts shares to recover mean utilities, and uses instruments to separate price variation from unobserved demand shocks.

2.2 Simulation design used to evaluate best practices

- **Purpose of the simulations.** The Monte Carlo exercises are not an empirical application in a single industry. They are designed to evaluate estimation practices under controlled conditions.
- **Baseline market structure.**
  - There are $T = 20$ markets.
  - The number of firms per market is randomly chosen from $\{2, 5, 10\}$.
  - Each firm produces a number of products randomly chosen from $\{3, 4, 5\}$.
  - Typical sample sizes are roughly $200 < N < 600$ product-market observations.
- **Demand primitives.**
  - Linear demand characteristics are $[1, x_{jt}, p_{jt}]$.
  - The simple model has one random coefficient on x_{jt}.

  - The complex model adds a random coefficient on price.
  - The RCNL model adds a nesting parameter $\rho = 0.5$.
- **Supply primitives.**
  - Supply characteristics are $[1, x_{jt}, w_{jt}]$.
  - Marginal cost is linear in these characteristics plus ω_{jt}.
  - The cost shifter is deliberately weak in the baseline, with relatively low correlation between p_{jt} and w_{jt}.
- **Equilibrium construction.** Rather than simply drawing prices, the authors solve for equilibrium prices and shares. This matters because markups are endogenous objects in the model.
- **Number of simulations.** For each configuration, the paper solves and estimates 1,000 synthetic datasets.

2.3 Replication exercises

- **Nevo (2000b) cereal example.** The first replication uses the public Nevo data. This problem includes demographics, unobserved heterogeneity, and product fixed effects.
- **BLP automobile example.** The second replication revisits the original automobile demand application. This problem includes a supply side and price coefficient heterogeneity.
- **Purpose.** These replications show that the numerical recommendations are not only Monte Carlo artifacts. They also matter in well-known empirical BLP examples.
- **Main replication message.** With tight tolerances, better integration, and feasible optimal instruments, the large dispersion across optimization routines emphasized in prior work mostly disappears.

2.4 Computational and diagnostic objects

- **Fixed-point diagnostics.** Number of contraction evaluations, convergence rate, and tolerance for the log share residual.
- **Optimization diagnostics.** Objective value, gradient norm, Hessian positive semidefiniteness, convergence rate, and stability across starting values.
- **Integration diagnostics.** Error in simulated shares, number of integration nodes, and sensitivity of estimates to integration rule.
- **Instrument diagnostics.** Bias, median absolute error, strength of cost shifters, and steepness of the profiled GMM objective.
- **Economic diagnostics.** Own-price elasticities, markups, merger effects, and welfare calculations.

3 Models and empirical specifications (all equations + interpretation)

3.1 Random-coefficients demand system

Consumers choose among products $j \in J_t$ and an outside option. Consumer i's indirect utility from product j in market t is

$$U_{ijt} = \delta_{jt} + \mu_{ijt} + \varepsilon_{ijt}. \tag{1}$$

Here:

- δ_{jt} is the product's mean utility,
- μ_{ijt} is consumer-specific taste heterogeneity,
- ε_{ijt} is the idiosyncratic logit error.

With type-I extreme value errors, the model-implied share is

$$s_{jt}(\delta_t, \tilde{\theta}_2) = \int \frac{\exp(\delta_{jt} + \mu_{ijt})}{\sum_{k \in J_t} \exp(\delta_{kt} + \mu_{ikt})} f(\mu_{it} \mid \tilde{\theta}_2)\, d\mu_{it}. \tag{2}$$

The BLP inversion solves

$$S_{jt} = s_{jt}(\delta_t, \tilde{\theta}_2)$$

for the vector δ_t in each market.

The mean utility equation is

$$\delta_{jt}(S_t, \tilde{\theta}_2) = [x_{jt}, v_{jt}]\beta - \alpha p_{jt} + \xi_{jt}. \tag{3}$$

Interpretation. The model uses market shares to infer the mean utility of each product. The price coefficient and random-coefficient parameters determine the mapping from shares to utilities. Once this mapping is done, the demand equation becomes a linear IV equation in the mean utility.

3.2 Supply side and markups

For a multi-product firm f, profits in market t are

$$\sum_{j \in J_{ft}} s_{jt}(p_t)(p_{jt} - c_{jt}).$$

The first-order conditions can be written in matrix form as

$$s_t(p_t) = \Delta_t(p_t)(p_t - c_t), \tag{4}$$

where

$$\Delta_t(p_t) = -H_t \odot \frac{\partial s_t}{\partial p_t}. \tag{5}$$

Thus markups are

$$\eta_t(p_t, s_t, \theta_2) = \Delta_t(p_t)^{-1} s_t(p_t), \tag{6}$$

and marginal costs are recovered as

$$c_t = p_t - \eta_t. \tag{7}$$

The marginal cost equation is

$$f_{MC}(p_{jt} - \eta_{jt}(\theta_2)) = [x_{jt}, w_{jt}]\gamma + \omega_{jt}. \tag{8}$$

Interpretation. The supply side converts demand derivatives into markups. Given prices, shares, ownership, and demand parameters, the researcher can back out marginal costs and impose cost-side moments. This can add a lot of information, but only if the conduct and marginal-cost specification are credible.

3.3 Stacked GMM estimator

The paper stacks demand and supply moments:

$$g(\theta) = \begin{pmatrix} \frac{1}{N}\sum_{j,t} \xi_{jt}(\theta) Z^D_{jt} \\ \frac{1}{N}\sum_{j,t} \omega_{jt}(\theta) Z^S_{jt} \end{pmatrix}. \tag{9}$$

The GMM objective is

$$\min_{\theta} q(\theta) = g(\theta)' W g(\theta). \tag{10}$$

Interpretation. Demand moments use instruments orthogonal to unobserved product quality. Supply moments use instruments orthogonal to unobserved marginal cost. Joint estimation can improve precision and identification because the same nonlinear demand parameters affect both equations.

3.4 Nested fixed point algorithm

For each candidate value of the nonlinear parameters θ_2, the estimator:

1. solves the share inversion $S_t = s_t(\delta_t, \theta_2)$ for $\hat{\delta}_t(\theta_2)$;
2. constructs demand derivatives and markups $\hat{\eta}_t(\theta_2)$;
3. backs out marginal costs;
4. runs a linear IV-GMM problem to concentrate out the linear demand and cost parameters;
5. evaluates the GMM objective in the nonlinear parameters.

The paper's useful reformulation defines

$$Y^D_{jt} \equiv \hat{\delta}_{jt}(\theta_2) + \alpha p_{jt} = [x_{jt}, v_{jt}]\beta + \xi_{jt}, \tag{11}$$

$$Y^S_{jt} \equiv \hat{c}_{jt}(\theta_2) = [x_{jt}, w_{jt}]\gamma + \omega_{jt}. \tag{12}$$

Stacking demand and supply gives a standard linear IV problem:

$$\begin{pmatrix} Y^D \\ Y^S \end{pmatrix} = \begin{pmatrix} X^D & 0 \\ 0 & X^S \end{pmatrix} \begin{pmatrix} \beta \\ \gamma \end{pmatrix} + \begin{pmatrix} \xi \\ \omega \end{pmatrix}. \tag{13}$$

Interpretation. This is the algebraic trick that lets high-dimensional fixed effects be absorbed even when supply and demand are estimated jointly. The nonlinear parameters move into the left-hand side; conditional on them, the remaining problem is linear.

3.5 Share inversion and SQUAREM

The standard BLP contraction is

$$\delta_t^{h+1} = \delta_t^h + \log S_t - \log s_t(\delta_t^h, \theta_2). \tag{14}$$

The target is a tight tolerance such as

$$\left\| \log S_t - \log s_t(\delta_t, \theta_2) \right\|_\infty \leq \varepsilon_{tol}. \tag{15}$$

The paper recommends a very tight inner-loop tolerance, typically around 10^{-14} to 10^{-12}, and uses SQUAREM acceleration as a default.

Interpretation. Loose fixed-point tolerances create numerical error in $\hat{\delta}_t$, which then contaminates the GMM objective. SQUAREM speeds up the contraction substantially while avoiding the full cost of Newton-style Jacobian calculations.

3.6 Nested logit and RCNL variant

For the random-coefficients nested logit model, the nesting parameter ρ changes the contraction. The dampened update is

$$\delta_t^{h+1} = \delta_t^h + (1-\rho)\left[\log S_t - \log s_t(\delta_t^h, \theta_2)\right]. \tag{16}$$

Interpretation. As $\rho \to 1$, the contraction becomes very slow. This is why the paper finds that Levenberg–Marquardt methods can be especially useful for RCNL problems.

3.7 Numerical integration over heterogeneity

The integral in the mixed logit share can be approximated with nodes and weights (ν_{it}, w_{it}):

$$s_{jt}(\delta_t, \theta_2) \approx s_{jt}(\delta_t, \theta_2; I_t) = \sum_{i \in I_t} w_{it}\, s_{ijt}(\delta_t, \mu_{it}(\nu_{it}, \theta_2)). \tag{17}$$

The paper compares:

- pseudo-Monte Carlo draws,
- modified Latin hypercube sampling,
- scrambled Halton draws,
- importance sampling,
- Gauss–Hermite product rules,
- sparse grids.

Interpretation. Poor integration creates simulation noise in the objective. In low dimensions, high-order product rules are recommended. In higher dimensions, Halton draws and sparse grids scale better.

3.8 Counterfactual pricing equilibria

Many BLP applications require solving for new prices after a merger or cost shock:

$$p_t = c_t + \eta_t(p_t, H_t^*). \tag{18}$$

The paper emphasizes that direct fixed-point iteration on this equation is not guaranteed to be a contraction. It recommends the Morrow–Skerlos ζ-markup approach. Decompose demand derivatives into a diagonal matrix Λ_t and dense matrix Γ_t:

$$\frac{\partial s_t}{\partial p_t} = \Lambda_t(p_t) - \Gamma_t(p_t). \tag{19}$$

Then iterate on

$$p_t \leftarrow c_t + \zeta_t(p_t), \tag{20}$$

where

$$\zeta_t(p_t) = \Lambda_t(p_t)^{-1}\left[H_t^* \odot \Gamma_t(p_t)\right](p_t - c_t) - \Lambda_t(p_t)^{-1} s_t(p_t). \tag{21}$$

Interpretation. This fixed point is designed for mixed-logit Bertrand pricing problems and is much more reliable than simply iterating on price equals cost plus markup. Reliable pricing solutions also make feasible optimal instruments computationally practical.

3.9 Feasible optimal instruments

Let the stacked structural errors be (ξ_{jt}, ω_{jt}) with covariance matrix Ω_t. The optimal-instrument logic uses the expected Jacobian of the structural errors:

$$Z_{jt}^{Opt} \approx E\left[D_{jt}(Z_t)\Omega_t^{-1} \mid Z_t\right], \tag{22}$$

where D_{jt} collects derivatives of (ξ_{jt}, ω_{jt}) with respect to the structural parameters.

The paper partitions these into demand and supply instruments:

$$Z_{jt}^{Opt,D}, \qquad Z_{jt}^{Opt,S}. \tag{23}$$

The practical algorithm is:

1. obtain an initial estimate;
2. estimate the covariance of structural errors;
3. simulate or approximate draws of (ξ^*, ω^*);
4. solve equilibrium prices and shares for each draw;
5. compute Jacobian terms;

6. average them to approximate the optimal instruments.

The paper implements three variants:

- **Approximate:** set $(\xi^*, \omega^*) = (0, 0)$;
- **Asymptotic:** draw from an estimated asymptotic normal distribution;
- **Empirical:** draw from the empirical distribution of residuals.

Interpretation. Optimal instruments are hard to construct because prices and shares are equilibrium objects. PyBLP makes this practical by combining analytic derivatives with fast pricing-equilibrium solvers.

3.10 Testing supply-side restrictions

If the supply side is misspecified, adding supply moments can hurt. The paper discusses testing supply-side moments with an overidentification-style comparison. A simple statistic is

$$LR = N\left[g(\hat{\theta})'Wg(\hat{\theta}) - g_D(\hat{\theta}_D)'W_D g_D(\hat{\theta}_D)\right] \sim \chi^2_{K-K_x}. \tag{24}$$

Interpretation. Correctly specified supply restrictions help, but conduct assumptions are not innocuous. The model can use overidentifying restrictions to test whether the extra supply-side moments are compatible with the demand-only estimates.

4 Identification and instruments (what makes the estimates credible)

4.1 Why price endogeneity is central

- Prices are endogenous because unobserved product quality ξ_{jt} affects both demand and equilibrium prices.
- A product with a high unobserved demand shock may have both higher market share and higher price.
- Estimating the demand equation without instruments would typically make demand appear less elastic than it really is.
- The BLP estimator therefore needs instruments that shift prices or markups without directly shifting unobserved demand.

4.2 Traditional BLP instruments

- Traditional BLP instruments use functions of own and rival product characteristics.
- The logic is that rival characteristics affect equilibrium markups and prices through competition, but are excluded from product j's unobserved demand shock.
- The paper considers several versions:

  - **Own instruments:** own exogenous characteristics and interactions;
  - **Sums instruments:** sums of characteristics of products owned by the same firm and by rival firms;
  - **Local differentiation instruments:** counts of products close in characteristic space;
  - **Quadratic differentiation instruments:** distances in characteristic space.
- The differentiation instruments follow Gandhi and Houde's idea that closeness in product space is especially relevant for substitution and markups.

4.3 Why optimal instruments matter

- The optimal instruments approximate the conditional expectation of the Jacobian of the structural errors.
- They are tailored to the nonlinear BLP problem rather than being generic functions of product characteristics.
- The Monte Carlo results show that optimal instruments reduce bias and median absolute error, especially for nonlinear taste parameters.
- The paper's preferred workflow is:
  1. start with differentiation instruments or sums of characteristics;
  2. get an initial estimate;
  3. construct feasible optimal instruments;
  4. re-estimate the model.

4.4 What the supply side identifies

- Supply moments impose that recovered marginal costs are related to cost shifters and exogenous characteristics.
- These moments help because demand parameters determine demand derivatives, which determine markups, which determine recovered costs.
- A correctly specified supply side provides cross-equation restrictions: the same nonlinear demand parameters must rationalize both demand-side shares and supply-side pricing behavior.
- This is especially useful when excluded cost shifters are weak. In the simulations, combining supply moments with optimal instruments can eliminate much of the bias in the price parameter.

4.5 Why supply-side misspecification is dangerous

- Supply moments are powerful only if the assumed conduct and marginal cost equation are credible.
- If the true pricing game is perfect competition, but the researcher estimates a multiproduct Bertrand model, the supply side can induce bias.

- The paper therefore does not say "always add supply." It says supply is valuable when correctly specified, and misspecification should be tested.
- The overidentification tests in PyBLP are useful because they can reject misspecified conduct assumptions in the Monte Carlo exercises.

4.6 Numerical identification versus economic identification

- The paper makes clear that numerical settings are not cosmetic.
- Weak instruments, noisy integration, loose inner-loop tolerances, and premature optimizer termination can flatten or distort the objective function.
- A flat objective looks like weak identification even if the economic model is theoretically identified.
- Best practices make the objective smoother and steeper near the optimum, which improves both optimization and inference.

5 Tables and figures

5.1 Table 1 and Algorithm 1: notation and the nested fixed point

Read:

- Table 1 is a notation map for the whole paper. The most important objects are observed shares S_{jt}, predicted shares s_{jt}, prices p_{jt}, mean utilities δ_{jt}, demand shocks ξ_{jt}, supply shocks ω_{jt}, ownership matrix H_t, markup η_{jt}, instruments Z, and GMM objective q.
- Algorithm 1 is the operational heart of the paper. For each nonlinear-parameter guess, solve for mean utilities, recover markups and costs, run linear IV-GMM, and evaluate the objective.
- The key innovation in the algorithm is the reformulation of the demand and supply equations into a stacked linear IV problem conditional on θ_2.
- This reformulation is what lets PyBLP absorb many fixed effects without building enormous dummy-variable matrices.

TABLE 1 Model Notation

j	Products	p_{jt}	Price
t	Markets	c_{jt}	Marginal cost
i	"Individuals"	x_{jt}	Exogenous product characteristic
f	Firms	v_{jt}	Excluded demand-shifter
h	Nesting groups	w_{jt}	Excluded supply-shifter
N	Set/number of products across all markets	U_{ijt}	Indirect utility
T	Set/number of markets	δ_{jt}	Mean utility
J_t	Set/number of products in market t	μ_{ijt}	Heterogeneous utility
I_t	Set/number of individuals in market t	ϵ_{ijt}	Idiosyncratic preference
F_t	Set/number of firms in market t	d_{ijt}	Choice indicator
J_{ft}	Set/number of products of firm f in market t	s_{ijt}	Choice probability
H	Set/number of nesting groups	s_{jt}	Calculated market share
J_{ht}	Set/number of products in nest h and market t	$\mathcal{S}_{jt}$	Observed market share
		ξ_{jt}	Demand-side structural error
θ	Parameters with dimension K		
θ_1	Linear demand-side parameters with dimension K_1	$\mathcal{H}_t$	Ownership or holdings matrix
θ_2	Nonlinear common parameters with dimension K_2	Δ_t	Intra-firm demand derivatives
θ_3	Linear supply-side parameters with dimension K_3	η_{jt}	Multi-product Bertrand markup
		ω_{jt}	Supply-side structural error
β	θ_1 excluding fixed effects		
$\widetilde{\theta}_2$	Parameters in θ_2 governing heterogeneity	Z	Instruments
α	Parameter in θ_2 on price	W	Weighting matrix
γ	θ_3 excluding fixed effects	g	Sample moments
ρ	Nesting parameter in θ_2	q	Objective function

TABLE 2 Various Forms of Instrumental Variables

$Z_{jt}^{Own} = \{1, x_{jt}, w_{jt}, x_{jt}^2, w_{jt}^2, x_{jt} \cdot w_{jt}\}$
$Z_{jt}^{Sums} = \{Z_{jt}^{Own}, \sum_{k \in J_{ft} \setminus \{j\}} 1, \sum_{k \notin J_{ft}} 1, \sum_{k \in J_{ft} \setminus \{j\}} x_{kt}, \sum_{k \notin J_{ft}} x_{kt}\}$
$Z_{jt}^{Local} = \{Z_{jt}^{Own}, \sum_{k \in J_{ft} \setminus \{j\}} 1(|d_{jkt}| < \text{SD}(d)), \sum_{k \notin J_{ft}} 1(|d_{jkt}| < \text{SD}(d))\}$
$Z_{jt}^{Quad} = \{Z_{jt}^{Own}, \sum_{k \in J_{ft} \setminus \{j\}} d_{jkt}^2, \sum_{k \notin J_{ft}} d_{jkt}^2\}$

Note: Optimal instruments, $Z_{jt}^{Opt,D}$ and $Z_{jt}^{Opt,D}$ in (31), are approximated with Algorithm 2 using initial estimates from a single GMM step under Z_{jt}^{Sums}. We define the difference in characteristic space from product j as $d_{jkt} = d_{kt} - d_{jt}$ for each characteristic in x_{jt}. For the Complex simulation we also include a measure of expected price $E[p_{jt}|Z_t]$ as an additional x_{jt}: fitted values from a linear regression of endogenous prices onto all exogenous variables, including the above instruments. For the RCNL simulation, we also include counts of products within the same nest in each market.

TABLE 3 Fixed Effect Absorption

Dimensions	Levels	Absorbed	Seconds	Megabytes
1	216	No	56	721
1	216	Yes	20	39
2	216 × 216	No	112	1414
2	216 × 216	Yes	25	43
2	36 × 1296	No	330	2498
2	36 × 1296	Yes	25	43
3	36 × 36 × 36	No	46	375
3	36 × 36 × 36	Yes	24	47

Note: This table documents the impact of fixed effect (FE) absorption on estimation speed and memory usage during one GMM step. Reported values are medians across 100 different simulations. When not absorbed, FEs are included as dummy variables. A single FE is absorbed with de-meaning; multiple, with iterative de-meaning, also known as the Method of Alternating Projections (MAP). To accommodate FEs in the Simple simulation, we first set $T = 6^4$ and $F_t = J_f = 6$ so that there are $N = 6^6 = 46,656$ products. We then randomly assign each $n \in \{1, \ldots, N\}$ to a product and add FEs drawn from the standard uniform distribution. The 1-D case has $\beta_{n \bmod 216}$. For the "square" 2-D case, we add $\beta_{n \text{ div } 216}$. The "uneven" 2-D case has $\beta_{n \bmod 36}$ and $\beta_{n \text{ div } 36}$. The 3-D case has $\beta_{n \bmod 36}$, $\beta_{n(\text{div } 36) \bmod 36}$, and $\beta_{n \text{div } 216}$.

5.2 Table 2: alternative instruments

Read:

- Table 2 lists the instrument families used in the Monte Carlo exercises.
- Own instruments use only own product characteristics and interactions.
- Sums instruments add sums of own-firm and rival product characteristics.
- Local and Quadratic instruments implement differentiation-IV logic: they measure product proximity in characteristic space.
- Optimal instruments are constructed after an initial GMM estimate. They are more computationally demanding but generally perform best.
- The table is useful because it clarifies that "BLP instruments" are not one object; they are a family of choices with different statistical strength.

5.3 Table 3: fixed-effect absorption

Read:

- Table 3 shows that absorbing fixed effects dramatically reduces memory and sometimes computation time.
- With one fixed-effect dimension of 216 levels, dummy variables require about **56 seconds** and **721 MB**; absorption uses about **20 seconds** and **39 MB**.
- With two fixed-effect dimensions of 216 × 216, dummy variables require about **112 seconds** and **1,414 MB**; absorption uses about **25 seconds** and **43 MB**.
- With uneven two-dimensional fixed effects of 36 × 1296, dummy variables require about **330 seconds** and **2,498 MB**; absorption uses about **25 seconds** and **43 MB**.
- The economic lesson is practical: rich product, store, or market fixed effects can be feasible in BLP estimation if the problem is written correctly.

5.4 Table 4: fixed-point algorithms

Read:

TABLE 4 Fixed Point Algorithms

Problem	Median s_0	Algorithm	Jacobian	Termination	Mean Milliseconds	Mean Evaluations	Percent Converged
Simple simulation ($\beta_1 = -7$)	0.91	Iteration	No	Absolute L^∞	5.95	41.85	100.00%
Simple simulation ($\beta_1 = -7$)	0.91	DF-SANE	No	Absolute L^∞	3.43	16.27	100.00%
Simple simulation ($\beta_1 = -7$)	0.91	SQUAREM	No	Absolute L^∞	2.56	15.94	100.00%
Simple simulation ($\beta_1 = -7$)	0.91	SQUAREM	No	Relative L^1	2.75	15.26	100.00%
Simple simulation ($\beta_1 = -7$)	0.91	Powell	Yes	Relative L^1	3.48	16.33	28.56%
Simple simulation ($\beta_1 = -7$)	0.91	LM	Yes	Relative L^1	2.31	8.91	100.00%
Simple simulation ($\beta_1 = -1$)	0.27	Iteration	No	Absolute L^∞	29.35	212.09	100.00%
Simple simulation ($\beta_1 = -1$)	0.27	DF-SANE	No	Absolute L^∞	7.10	35.28	100.00%
Simple simulation ($\beta_1 = -1$)	0.27	SQUAREM	No	Absolute L^∞	5.40	34.58	100.00%
Simple simulation ($\beta_1 = -1$)	0.27	SQUAREM	No	Relative L^1	5.73	33.91	100.00%
Simple simulation ($\beta_1 = -1$)	0.27	Powell	Yes	Relative L^1	3.67	17.21	11.38%
Simple simulation ($\beta_1 = -1$)	0.27	LM	Yes	Relative L^1	2.35	8.92	100.00%
Complex simulation ($\beta_1 = -7$)	0.91	Iteration	No	Absolute L^∞	8.35	45.02	100.00%
Complex simulation ($\beta_1 = -7$)	0.91	DF-SANE	No	Absolute L^∞	4.52	17.67	100.00%
Complex simulation ($\beta_1 = -7$)	0.91	SQUAREM	No	Absolute L^∞	3.32	16.14	100.00%
Complex simulation ($\beta_1 = -7$)	0.91	SQUAREM	No	Relative L^1	3.50	15.50	100.00%
Complex simulation ($\beta_1 = -7$)	0.91	Powell	Yes	Relative L^1	4.03	14.89	32.29%
Complex simulation ($\beta_1 = -7$)	0.91	LM	Yes	Relative L^1	2.85	8.98	100.00%
Complex simulation ($\beta_1 = -1$)	0.28	Iteration	No	Absolute L^∞	39.35	216.80	100.00%
Complex simulation ($\beta_1 = -1$)	0.28	DF-SANE	No	Absolute L^∞	8.92	36.23	100.00%
Complex simulation ($\beta_1 = -1$)	0.28	SQUAREM	No	Absolute L^∞	7.03	34.75	100.00%
Complex simulation ($\beta_1 = -1$)	0.28	SQUAREM	No	Relative L^1	7.51	34.09	100.00%
Complex simulation ($\beta_1 = -1$)	0.28	Powell	Yes	Relative L^1	4.80	17.70	9.71%
Complex simulation ($\beta_1 = -1$)	0.28	LM	Yes	Relative L^1	2.90	8.93	100.00%
RCNL simulation ($\rho = 0.5$)	0.92	Iteration	No	Absolute L^∞	21.63	93.40	100.00%
RCNL simulation ($\rho = 0.5$)	0.92	DF-SANE	No	Absolute L^∞	9.62	32.30	100.00%
RCNL simulation ($\rho = 0.5$)	0.92	SQUAREM	No	Absolute L^∞	8.45	33.49	100.00%
RCNL simulation ($\rho = 0.5$)	0.92	SQUAREM	No	Relative L^1	8.53	31.33	100.00%
RCNL simulation ($\rho = 0.5$)	0.92	Powell	Yes	Relative L^1	4.67	14.44	60.65%
RCNL simulation ($\rho = 0.5$)	0.92	LM	Yes	Relative L^1	3.27	8.89	100.00%
RCNL simulation ($\rho = 0.8$)	0.92	Iteration	No	Absolute L^∞	58.01	250.34	100.00%
RCNL simulation ($\rho = 0.8$)	0.92	DF-SANE	No	Absolute L^∞	16.30	55.50	99.92%
RCNL simulation ($\rho = 0.8$)	0.92	SQUAREM	No	Absolute L^∞	14.04	55.54	100.00%
RCNL simulation ($\rho = 0.8$)	0.92	SQUAREM	No	Relative L^1	13.95	51.70	100.00%
RCNL simulation ($\rho = 0.8$)	0.92	Powell	Yes	Relative L^1	6.48	20.16	61.48%
RCNL simulation ($\rho = 0.8$)	0.92	LM	Yes	Relative L^1	3.62	9.64	100.00%
NEVO example	0.54	Iteration	No	Absolute L^∞	12.95	86.64	100.00%
NEVO example	0.54	DF-SANE	No	Absolute L^∞	5.50	25.40	100.00%
NEVO example	0.54	SQUAREM	No	Absolute L^∞	4.05	24.06	100.00%
NEVO example	0.54	SQUAREM	No	Relative L^1	4.26	22.80	100.00%
NEVO example	0.54	Powell	Yes	Relative L^1	3.84	17.06	29.20%
NEVO example	0.54	LM	Yes	Relative L^1	2.62	9.34	100.00%
BLP example	0.89	Iteration	No	Absolute L^∞	152.72	203.04	100.00%
BLP example	0.89	DF-SANE	No	Absolute L^∞	34.93	42.37	100.00%
BLP example	0.89	SQUAREM	No	Absolute L^∞	31.54	40.61	100.00%
BLP example	0.89	SQUAREM	No	Relative L^1	31.43	39.38	100.00%
BLP example	0.89	Powell	Yes	Relative L^1	26.40	19.31	9.34%
BLP example	0.89	LM	Yes	Relative L^1	17.64	8.71	100.00%

Note: This table documents the impact of algorithm choice on solving the nested fixed point. Reported values are medians across 100 different simulations and 10 identical runs of the example problems. We report the number of milliseconds and contraction evaluations needed to solve the fixed point, averaged across all markets and one GMM step's objective evaluations. We also report each algorithm's convergence rate: the percent of times no numerical errors were encountered and a limit of 1000 iterations was not reached. We configure simple iteration with an absolute L^∞ norm tolerance of 1E−14 and compare it with DF-SANE and SQUAREM. The MINPACK (More, Garbow, and Hillstrom, 1980) implementations of algorithms that do require a Jacobian—a modification of the Powell hybrid method and Levenberg–Marquardt (LM)—only support relative L^2 norm tolerances, so for comparison's sake we also include SQUAREM with the same termination condition. Simulations are configured as described in Section 5, except for the coefficient on the constant term, β_0, and the nesting parameter, ρ, which we vary to document the effects of decreasing the outside share s_0 and of dampening the contraction in the RCNL model. The example problems from Nevo (2000b) and Berry, Levinsohn, and Pakes (1995, 1999) are the replications described in Section 6.

- Table 4 compares standard fixed-point iteration, DF-SANE, SQUAREM, Powell, and Levenberg–Marquardt.
- SQUAREM is the main workhorse recommendation. It cuts fixed-point evaluations by roughly **3–8 times** relative to simple iteration.
- In the simple simulation with a small outside share, simple iteration takes about **29.35 ms** and **212** evaluations, while SQUAREM takes about **5.40 ms** and **35** evaluations.
- Levenberg–Marquardt often uses even fewer evaluations, and is especially useful for RCNL problems, but each iteration is more expensive.
- Powell's method has low convergence rates at tight tolerances, so it is not recommended as the default.
- The main message is that the inner-loop solver is economically important because bad or slow inversion can make the whole estimation procedure unreliable.

5.5 Figure 1: numerical integration error

Read:

- Figure 1 compares integration methods by root mean squared error in predicted market shares.
- Pseudo-Monte Carlo improves slowly, at the familiar $I_t^{-1/2}$ rate.
- Scrambled Halton and MLHS improve over plain Monte Carlo.
- Product-rule quadrature is very accurate in low dimensions but faces the curse of dimensionality.
- Sparse grids perform well as the number of random coefficients rises.
- The practical lesson is: do not treat simulation draws as an afterthought. Bad integration can create noisy objectives, unstable estimates, and misleading optimization results.

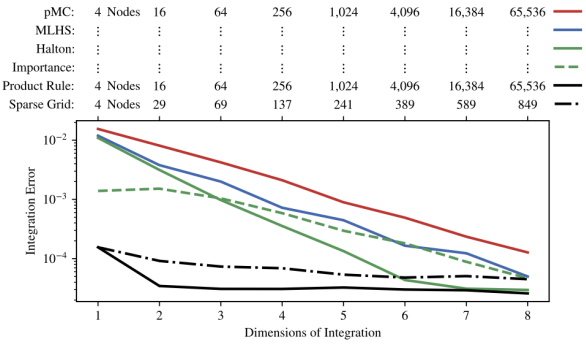
5.6 Table 5 and Figures 2–3: instruments and supply restrictions

Read:

- Table 5 is one of the paper's main Monte Carlo tables. It compares Own, Sums, Local, Quadratic, and Optimal instruments, with and without supply moments.

FIGURE 1

INTEGRATION ERROR [Color figure can be viewed at wileyonlinelibrary.com]



This plot documents the performance of different numerical integration methods in terms of root mean square error of market shares. Reported values are medians across 100 different markets and are on a logarithmic scale. To document the curse of dimensionality, as we increase the number of random coefficients K_2, we also increase the number of integration nodes I_t to match that of a Gauss-Hermite product rule that exactly integrates polynomials of degree 7 or less: $I_t = 4^{K_2}$. To accommodate $K_2 > 1$ dimensions of integration in the Simple simulation, we draw additional exogenous characteristics from the standard uniform distribution. On these new characteristics, we add uncorrelated and mean-zero random coefficients. For comparability's sake, we use the unaltered Simple simulation's market shares $\mathcal{S}_t$ for all K_2, and set each random coefficient's variance to $1/K_2$ so that the distribution of utility is invariant to K_2. We use 1 million pseudo-Monte Carlo (pMC) draws to precisely compute $\boldsymbol{\delta}_t = D_t^{-1}(\boldsymbol{\mathcal{S}}_t, \widetilde{\theta}_2)$. With this, we compute the integration error $||\boldsymbol{\mathcal{S}}_t - \boldsymbol{s}_t(\boldsymbol{\delta}_t, \widetilde{\theta}_2; I_t)||_2$ of $s_{jt}(\boldsymbol{\delta}_t, \widetilde{\theta}_2; I_t)$ from (24) under various integration methods and nodes I_t. In addition to a less-precise pMC rule, we also consider Modified Latin Hypercube Sampling (MLHS); Halton draws where in each dimension we use a different prime (2, 3, etc.), discard the first 1000 points, and scramble the sequence according to the recipe in Owen (2017); importance sampling of Halton draws according to the Berry, Levinsohn, and Pakes (1995) procedure described in Section 3 with $\widetilde{\theta}_2$ and $\boldsymbol{\delta}_t$ are replaced by their true values; a Gauss-Hermite product rule that exactly integrates polynomials of degree 7 or less; and sparse grids with the same polynomial order (Heiss and Winschel, 2008).

TABLE 5 Alternative Instruments

			True Value				Median Bias				Median Absolute Error			
Simulation	Supply	Instruments	Seconds	α	σ_x	σ_ξ	ρ	α	σ_x	σ_ξ	ρ	α	σ_x	σ_ξ	ρ
Simple	No	Own	0.6	−1	3			0.126	−0.045			0.238	0.257		
Simple	No	Sums	0.6	−1	3			0.224	−0.076			0.257	0.208		
Simple	No	Local	0.6	−1	3			0.181	−0.056			0.242	0.235		
Simple	No	Quadratic	0.6	−1	3			0.206	−0.085			0.263	0.239		
Simple	No	Optimal	0.8	−1	3			0.218	−0.049			0.250	0.174		
Simple	Yes	Own	1.4	−1	3			0.021	0.006			0.226	0.250		
Simple	Yes	Sums	1.5	−1	3			0.054	−0.020			0.193	0.196		
Simple	Yes	Local	1.4	−1	3			0.035	−0.006			0.207	0.229		
Simple	Yes	Quadratic	1.4	−1	3			0.047	−0.022			0.217	0.237		
Simple	Yes	Optimal	2.2	−1	3			0.005	0.012			0.170	0.171		
Complex	No	Own	1.1	−1	3	0.2		−0.025	0.000	−0.200		0.381	0.272	0.200	
Complex	No	Sums	1.1	−1	3	0.2		0.225	−0.132	−0.057		0.263	0.217	0.200	
Complex	No	Local	1.0	−1	3	0.2		0.184	−0.107	−0.085		0.274	0.236	0.200	
Complex	No	Quadratic	1.0	−1	3	0.2		0.200	−0.117	−0.198		0.299	0.243	0.200	
Complex	No	Optimal	1.6	−1	3	0.2		0.191	−0.119	0.001		0.274	0.195	0.200	
Complex	Yes	Own	3.9	−1	3	0.2		−0.213	0.060	0.208		0.325	0.263	0.208	
Complex	Yes	Sums	3.3	−1	3	0.2		0.018	−0.104	0.052		0.203	0.207	0.180	
Complex	Yes	Local	3.4	−1	3	0.2		−0.043	−0.078	0.135		0.216	0.225	0.200	
Complex	Yes	Quadratic	3.5	−1	3	0.2		−0.028	−0.067	0.116		0.237	0.227	0.200	
Complex	Yes	Optimal	4.9	−1	3	0.2		−0.024	−0.036	−0.002		0.193	0.171	0.191	
RCNL	No	Own	5.1	−1	3		0.5	0.423	−1.235		0.199	0.463	1.393		0.218
RCNL	No	Sums	3.4	−1	3		0.5	0.222	−0.187		0.013	0.237	0.300		0.034
RCNL	No	Local	3.4	−1	3		0.5	0.216	−0.194		0.016	0.247	0.324		0.039
RCNL	No	Quadratic	3.4	−1	3		0.5	0.211	−0.231		0.018	0.252	0.354		0.042
RCNL	No	Optimal	4.3	−1	3		0.5	0.217	−0.031		−0.008	0.230	0.155		0.021
RCNL	Yes	Own	9.7	−1	3		0.5	0.205	−0.955		0.162	0.301	1.201		0.189
RCNL	Yes	Sums	7.0	−1	3		0.5	0.047	−0.154		0.018	0.148	0.275		0.034
RCNL	Yes	Local	7.1	−1	3		0.5	0.038	−0.148		0.020	0.172	0.298		0.039
RCNL	Yes	Quadratic	7.1	−1	3		0.5	0.036	−0.169		0.022	0.168	0.330		0.042
RCNL	Yes	Optimal	10.0	−1	3		0.5	0.008	−0.003		0.002	0.111	0.136		0.017

Note: This table documents bias and variance of parameter estimates over 1000 simulated datasets for different instruments. Own instruments are $\{1, x_{jt}, w_{jt}, x_{jt}^2, w_{jt}^2, x_{jt} \cdot w_{jt}\}$. Sums include own and competitor product characteristics. Local and Quadratic instruments follow the definitions in Gandhi and Houde (2019). Optimal instruments are the "approximate" version from Algorithm 2. All instruments are defined in Table 2. For all problems, we use a Gauss-Hermite product rule that exactly integrates polynomials of degree 17 or less.

- In the simple model with supply, using optimal instruments gives median bias in α of about **0.005**, much smaller than many nonoptimal alternatives.
- In the RCNL model with supply, optimal instruments perform especially well: median biases are near zero for α, σ_x, and ρ.
- Figure 2 shows that the value of supply restrictions is largest when the excluded cost shifter is weak. With a correctly specified supply side, supply moments help eliminate bias. With a misspecified supply side, they can make estimates worse.
- Figure 3 profiles the GMM objective. Optimal instruments and supply moments make the objective steeper around the minimum, which helps both identification and numerical optimization.
- The table and figures together support the paper's preferred two-step workflow: start with differentiation or sums instruments, then construct feasible optimal instruments.

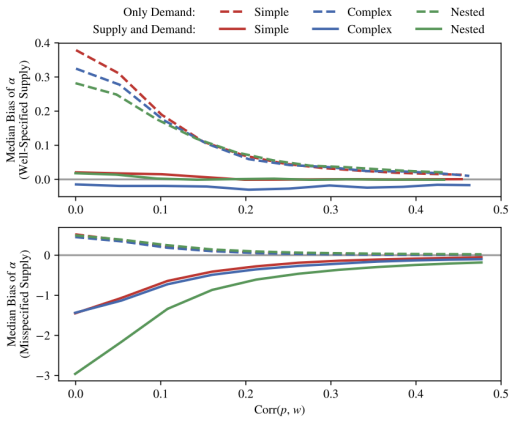
5.7 Figure 4 and Table 6: problem size and optimization

Read:

- Figure 4 shows that increasing the number of markets improves bias and median absolute error for both the price coefficient and random-coefficient parameters.
- For small samples, the demand-only estimator can have substantial bias in α; with more markets, demand-only and supply-demand specifications become more similar.
- The paper finds that around $T = 40$ markets can already be enough for reasonable performance in their demand-only simulations, especially with optimal instruments.
- Table 6 compares optimization algorithms. Most derivative-based optimizers converge reliably; derivative-free Nelder–Mead performs much worse.

FIGURE 2

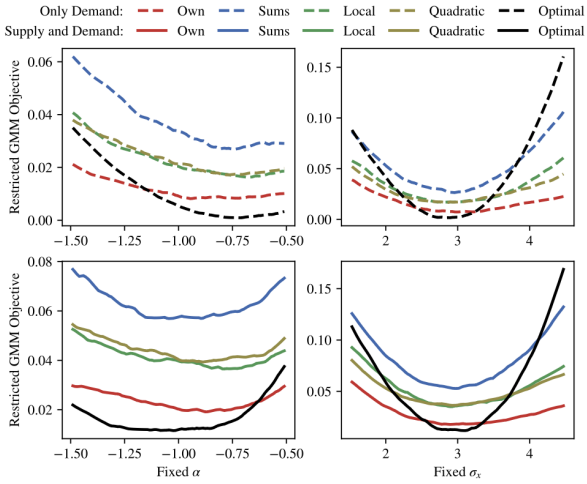
INSTRUMENT STRENGTH AND MISSPECIFICATION [Color figure can be viewed at wileyonlinelibrary.com]



Each plot documents how bias of the linear parameter on price, α, decreases with the strength of the cost shifter w_{jt}, which is included as a demand-side instrument. To weaken or strengthen the instrument, we vary its supply-side parameter from $\gamma_w = 0$ to $\gamma_w = 1$, and report the correlation this induces between w_{jt} and prices p_{jt}. Reported bias values are medians across 1000 different simulations. The top plot reports results for the simulation configurations described in Section 5. In the bottom plot, we simulate data according to perfect competition (i.e., prices are set equal to marginal costs instead of those that satisfy Bertrand-Nash first order conditions), but continue to estimate the model under the assumption of imperfect competition. For all problems, we use the "approximate" version of the feasible optimal instruments and a Gauss-Hermite product rule that exactly integrates polynomials of degree 17 or less.

FIGURE 3

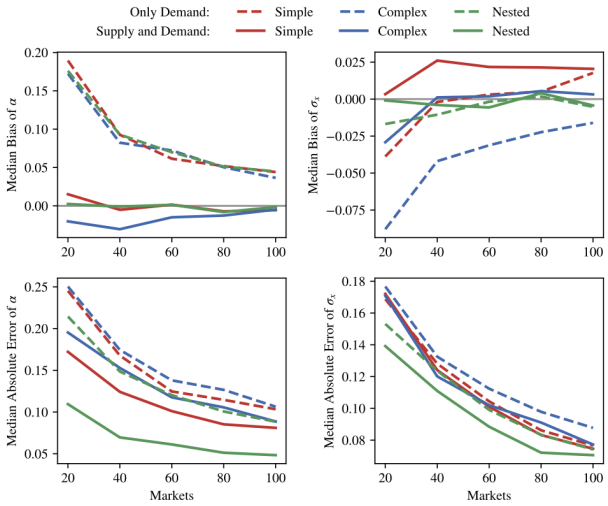
PROFILED GMM OBJECTIVE WITH ALTERNATIVE INSTRUMENTS [Color figure can be viewed at wileyonlinelibrary.com]



Each plot profiles the GMM objective with respect to a single parameter for the Simple simulation. We fix either α or σ_x, re-optimize over other parameters, and plot median restricted objective values over 100 different simulations. The left column profiles the objective over the price parameter α, whereas the right column profiles over the random coefficient σ_x. The top row uses moments from demand alone, whereas the bottom row uses both supply and demand moments. Own instruments are $[1, x_{jt}, w_{jt}, x_{jt}^2, w_{jt}^2, x_{jt} \cdot w_{jt}]$. Sums include own and competitor product characteristics. Local and Quadratic instruments follow the definitions in Gandhi and Houde (2019). Optimal instruments are the "approximate" version from Algorithm 2. All instruments are defined in Table 2. For all problems, we use a Gauss-Hermite product rule that exactly integrates polynomials of degree 17 or less.

FIGURE 4

PROBLEM SCALING [Color figure can be viewed at wileyonlinelibrary.com]



These plots document how bias and variance of parameter estimates decrease with the number of markets T. The top row plots median bias across 1000 simulations, whereas the bottom row plots median absolute error across the same simulations. The left column reports results for α and the right column reports results for σ_x. For all problems, we use the "approximate" version of the feasible optimal instruments and a Gauss-Hermite product rule that exactly integrates polynomials of degree 17 or less.

- Preferred algorithms include Knitro Interior/Direct and BFGS-style routines such as SciPy's L-BFGS-B.
- The recommendation is to use gradient-based optimizers, tight tolerances, box constraints, and multiple starting values or optimizers as checks.

TABLE 6 Optimization Algorithms

							Percent of Runs		Median, First GMM Step			
Simulation	Supply	$\lvert\theta_2\rvert$	Software	Algorithm	Gradient	Termination	Converged	PSD Hessian	Seconds	Evaluations	$q = \bar{g}'W\bar{g}$	$\lVert\nabla q\rVert_\infty$
Simple	No	1	Knitro	Interior/Direct	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	0.2	4	1.10E-08	8.30E-07
Simple	No	1	SciPy	L-BFGS-B	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	0.2	4	8.19E-09	7.28E-07
Simple	No	1	SciPy	BFGS	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	0.6	11	1.58E-08	1.03E-06
Simple	No	1	SciPy	TNC	Yes	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	99.9%	99.8%	0.6	10	3.89E-24	9.61E-15
Simple	No	1	SciPy	Nelder-Mead	No	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	66.5%	100.0%	19.6	115	1.08E-24	4.70E-15
Simple	Yes	2	Knitro	Interior/Direct	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	0.6	5	2.17E-06	3.54E-06
Simple	Yes	2	SciPy	L-BFGS-B	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	0.4	4	2.18E-06	3.32E-06
Simple	Yes	2	SciPy	BFGS	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	2.0	11	2.61E-06	5.26E-06
Simple	Yes	2	SciPy	TNC	Yes	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	99.8%	100.0%	2.2	18	2.15E-06	5.12E-11
Simple	Yes	2	SciPy	Nelder-Mead	No	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	53.3%	100.0%	31.7	251	2.14E-06	9.69E-13
Complex	No	3	Knitro	Interior/Direct	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	96.9%	0.7	6	1.92E-07	4.11E-06
Complex	No	3	SciPy	L-BFGS-B	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	93.6%	0.4	6	1.83E-07	3.59E-06
Complex	No	3	SciPy	BFGS	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	96.5%	1.9	26	2.25E-07	5.14E-06
Complex	No	3	SciPy	TNC	Yes	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	100.0%	86.9%	1.8	20	5.08E-20	1.61E-12
Complex	No	3	SciPy	Nelder-Mead	No	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	54.0%	74.6%	30.1	275	1.18E-24	1.39E-14
Complex	Yes	4	Knitro	Interior/Direct	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	93.7%	1.9	9	3.20E-06	6.11E-06
Complex	Yes	4	SciPy	L-BFGS-B	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	94.1%	1.4	9	3.12E-06	5.57E-06
Complex	Yes	4	SciPy	BFGS	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	94.2%	4.6	28	3.19E-06	6.36E-06
Complex	Yes	4	SciPy	TNC	Yes	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	99.5%	99.5%	5.7	31	2.87E-06	4.02E-10
Complex	Yes	4	SciPy	Nelder-Mead	No	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	45.5%	99.5%	64.6	480	2.80E-06	1.83E-12
RCNL	No	2	Knitro	Interior/Direct	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	1.7	10	1.67E-08	5.72E-06
RCNL	No	2	SciPy	L-BFGS-B	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	99.9%	1.9	11	1.52E-09	1.47E-06
RCNL	No	2	SciPy	BFGS	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	4.3	25	2.64E-09	1.95E-06
RCNL	No	2	SciPy	TNC	Yes	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	100.0%	99.6%	4.1	22	3.56E-19	1.16E-11
RCNL	No	2	SciPy	Nelder-Mead	No	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	56.4%	98.7%	60.5	243	1.27E-25	2.53E-14
RCNL	Yes	3	Knitro	Interior/Direct	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	3.4	13	2.83E-06	9.95E-06
RCNL	Yes	3	SciPy	L-BFGS-B	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	3.2	12	2.66E-06	3.85E-06
RCNL	Yes	3	SciPy	BFGS	Yes	$\lVert\nabla q\rVert_\infty$	100.0%	100.0%	9.3	37	2.73E-06	4.41E-06
RCNL	Yes	3	SciPy	TNC	Yes	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	100.0%	100.0%	7.2	24	2.95E-06	2.24E-09
RCNL	Yes	3	SciPy	Nelder-Mead	No	$\lVert\theta_2^s - \theta_2^{s-1}\rVert_\infty$	39.0%	100.0%	115.2	423	2.93E-06	4.60E-12

(*Continued*)

TABLE 6 Continued

Note: This table documents optimization convergence statistics over 1000 simulated datasets for different optimization algorithms. For comparison's sake, we only perform one GMM step and do not set parameter bounds. We report each algorithm's convergence rate (the percent of times the algorithm reported that it successfully found an optimum before reaching 1000 iterations) and the percent of times the final Hessian matrix was positive semidefinite. We also report medians for the number of seconds needed to solve each problem, the number of objective evaluations, the final GMM objective value, and the L^∞ norm of the gradient. We configure three algorithms to terminate with gradient-based L^∞ norms of 1E-5: Interior/Direct from Knitro, along with L-BFGS-B (limited-memory BFGS) and BFGS from SciPy. Because SciPy's derivative-free Nelder-Mead algorithm does not support gradient-based termination, we instead configure it to terminate with an absolute parameter-based L^∞ norm of 1E-5 and for comparison's sake use the same configuration for SciPy's TNC (truncated Newton) algorithm. For all problems, we use the "approximate" version of the feasible optimal instruments and a Gauss-Hermite product rule that exactly integrates polynomials of degree 17 or less.

5.8 Figures 5–6 and Tables 7–8: replication exercises

Read:

- Figure 5 shows how little code is needed to replicate the Nevo cereal problem in PyBLP. It emphasizes that PyBLP is not only a methodological proposal but also a usable software implementation.
- Table 7 reports the Nevo replication. The replication closely matches the published estimates, and best practices greatly reduce the scaled GMM objective.
- The mean own-price elasticity in the Nevo replication remains stable around **-3.6 to -3.7** across configurations, even though some nearly collinear demographic interaction terms move around.
- Figure 6 shows how to formulate the original BLP automobile problem through PyBLP, including access from R through `reticulate`.
- Table 8 reports the BLP automobile replication. Best practices use **10,000 scrambled Halton draws** and feasible optimal instruments.
- The BLP best-practices specification has mean own-price elasticity about **-3.461** and mean markup about **0.346**.
- The replication exercises show that PyBLP can reproduce canonical examples while making numerical choices explicit and easier to audit.

5.9 Figure 7: local optima and best practices

Read:

- Figure 7 revisits the Knittel–Metaxoglou concern that BLP estimation may produce many local optima and wide dispersion in elasticities.
- With loose optimization tolerances, PyBLP can replicate the dispersion emphasized in that literature.

FIGURE 5

NEVO (2000b) REPLICATION CODE [Color figure can be viewed at wileyonlinelibrary.com]

```python
import numpy as np
import pandas as pd

import pyblp

problem = pyblp.Problem(
    product_formulations=(
        pyblp.Formulation('0 + prices', absorb='C(product_ids)'),                    # Linear demand
        pyblp.Formulation('1 + prices + sugar + mushy'),                             # Nonlinear demand
    ),
    agent_formulation=pyblp.Formulation('0 + income + income_squared + age + child'),  # Demographics
    product_data=pd.read_csv(pyblp.data.NEVO_PRODUCTS_LOCATION),
    agent_data=pd.read_csv(pyblp.data.NEVO_AGENTS_LOCATION)
)

results = problem.solve(
    sigma=np.diag([0.3302, 2.4526, 0.0163, 0.2441]),         # Starting values for unobserved heterogeneity
    pi=[
        [ 5.4819,  0,      0.2037, 0     ],                  # Starting values for observed heterogeneity
        [15.8935, -1.2000, 0,      2.6342],
        [-0.2506,  0,      0.0511, 0     ],
        [ 1.2650,  0,     -0.8091, 0     ]
    ],
    method='1s',                                             # One-step GMM
    optimization=pyblp.Optimization('bfgs', {'gtol': 1e-5})  # Gradient-based termination tolerance
)

elasticities = results.compute_elasticities()
markups = results.compute_markups()
```

This Python code demonstrates how to construct and solve the problem from Nevo (2000b) in PyBLP. Names in the formulation objects correspond to variable names in the datasets, which are packaged with PyBLP and in this example are loaded into memory with the Python package `pandas`. Although not an explicit dependency of PyBLP, `pandas` is a convenient package for loading data into data frames, and comes pre-packaged in Anaconda installations. In addition to `pandas` data frames, PyBLP can also handle other data types such as NumPy structured arrays or simple dictionaries. Most estimation outputs are stored as attributes of the problem results class. Post-estimation outputs such as elasticities and markups can be computed with class methods. Estimation results are reported in Table 7.

FIGURE 6

BERRY, LEVINSOHN, AND PAKES (1995, 1999) REPLICATION CODE [Color figure can be viewed at wileyonlinelibrary.com]

```r
library(readr)
library(reticulate)

pyblp <- import('pyblp')

problem <- pyblp$Problem(
    product_formulations = tuple(
        pyblp$Formulation('1 + hpwt + air + mpd + space'),                          # Linear demand
        pyblp$Formulation('1 + prices + hpwt + air + mpd + space'),                 # Nonlinear demand
        pyblp$Formulation('1 + log(hpwt) + air + log(mpg) + log(space) + trend')    # Supply
    ),
    agent_formulation = pyblp$Formulation('0 + I(1 / income)'),                     # Price interaction
    costs_type = 'log',                                                             # Log-linear costs
    product_data = read_csv(pyblp$data$BLP_PRODUCTS_LOCATION),
    agent_data = read_csv(pyblp$data$BLP_AGENTS_LOCATION)
)

results <- problem$solve(
     sigma = diag(c(3.612, 0, 4.628, 1.818, 1.050, 2.056)),  # Starting values for unobserved heterogeneity
     pi = rbind(0, -43.501, 0, 0, 0, 0),                     # Starting value for the term on price
     initial_update = TRUE,                                  # Update weight matrix at starting values
     costs_bounds = tuple(0.001, NULL),                      # Constrain marginal costs to be positive
     W_type = 'clustered',                                   # Cluster by automobile model
     se_type = 'clustered'
)

elasticities = results$compute_elasticities()
markups = results$compute_markups()

instrument_results = results$compute_optimal_instruments(method = 'approximate')
updated_problem = instrument_results$to_problem()
```

This R code demonstrates how to construct and solve the problem from Berry, Levinsohn, and Pakes (1995, 1999) in PyBLP. Python functions are called with the R package `reticulate`. Similar packages that allow for Python interoperability are available in many other languages. Names in the formulation objects correspond to variable names in the datasets, which are are packaged with PyBLP and in this example are loaded into memory with the R package `readr`. Most estimation outputs are stored as attributes of the problem results class. Post-estimation outputs such as elasticities and markups can be computed with class methods. Here, the "approximate" technique from Algorithm 2 is used to construct feasible optimal instruments. This gives an optimal instruments results object that is converted into an updated problem, which can be solved like any other. Estimation results are reported in Table 8.

TABLE 7 Nevo (2000b) Replication

		Published Estimates	Replication	Tighter Tolerance	Best Practices
Means	Price	−32.433	−32.404	−62.729	−27.489
		(7.743)	(7.729)	(14.803)	(4.383)
Standard Deviations	Price	1.848	1.851	3.313	2.910
		(1.075)	(1.070)	(1.340)	(0.669)
	Constant	0.377	0.376	0.558	0.196
		(0.129)	(0.129)	(0.163)	(0.085)
	Sugar	0.004	0.003	0.006	0.028
		(0.012)	(0.012)	(0.014)	(0.008)
	Mushy	0.081	0.080	0.093	0.324
		(0.205)	(0.204)	(0.185)	(0.110)
Interactions	Price × income	16.598	16.457	588.318	15.957
		(172.334)	(172.237)	(270.441)	(98.164)
	Price × income squared	−0.659	−0.655	−30.192	−1.282
		(8.955)	(8.951)	(14.101)	(5.119)
	Price × child	11.625	11.543	11.054	4.551
		(5.207)	(5.166)	(4.123)	(2.405)
	Constant × income	3.089	3.100	2.292	6.253
		(1.213)	(1.203)	(1.209)	(0.541)
	Constant × age	1.186	1.172	1.284	0.162
		(1.016)	(1.001)	(0.631)	(0.207)
	Sugar × income	−0.193	−0.193	−0.385	−0.289
		(0.005)	(0.045)	(0.121)	(0.037)
	Sugar × age	0.029	0.030	0.052	0.046
		(0.036)	(0.036)	(0.026)	(0.014)
	Mushy × income	1.468	1.462	0.748	0.998
		(0.697)	(0.693)	(0.802)	(0.303)
	Mushy × age	−1.514	−1.502	−1.353	−0.523
		(1.103)	(1.091)	(0.667)	(0.188)
Mean own-price elasticity			−3.700	−3.618	−3.685
Mean markup			0.360	0.364	0.363
GMM objective		6.60E-03	6.61E-03	2.02E-03	2.03E-04
GMM objective scaled by N		1.49E+01	1.49E+01	4.56E+00	4.59E-01

Note: This table reports replication results for the model of Nevo (2000b) described in Section 6. From left to right, we report estimates from the original article, our replication results, additional results for when we reduce the BFGS optimization algorithm's gradient-based L^∞ norm from `1E-4` to a slightly tighter `1E-5`, and a final set of results using best estimation practices: a tighter termination tolerance and the "approximate" version of the feasible optimal instruments. Standard errors are in parentheses.

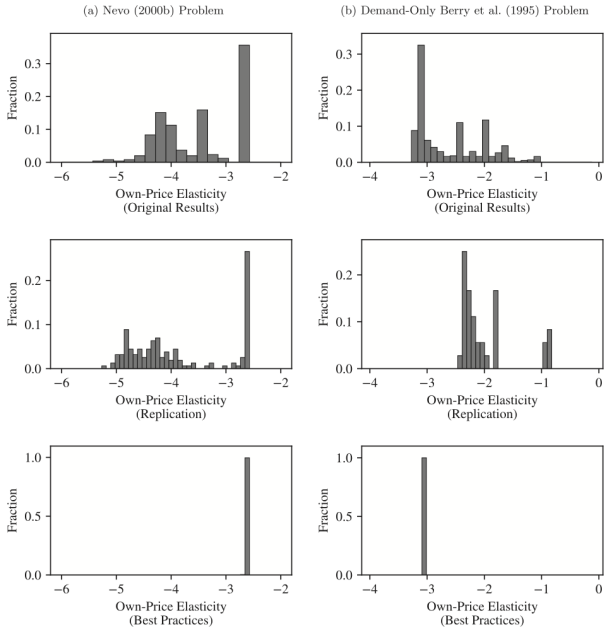
TABLE 8 Berry, Levinsohn, and Pakes (1995, 1999) Replication

		Published Estimates	Replication	Best Practices
Means	Constant	−7.061	−7.284	−6.679
		(0.941)	(2.807)	(1.304)
	HP/weight	2.883	3.460	2.774
		(2.019)	(1.415)	(0.833)
	Air	1.521	−0.999	0.572
		(0.891)	(2.101)	(0.349)
	MP$	−0.122	0.421	0.340
		(0.320)	(0.250)	(0.098)
	Size	3.460	4.178	3.920
		(0.610)	(0.658)	(0.322)
Standard deviations	Constant	3.612	2.025	2.962
		(1.485)	(6.065)	(1.637)
	HP/weight	4.628	6.101	1.388
		(1.885)	(2.200)	(2.107)
	Air	1.818	3.956	1.424
		(1.695)	(2.110)	(0.435)
	MP$	1.050	0.254	0.072
		(0.272)	(0.549)	(1.002)
	Size	2.056	1.908	0.231
		(0.585)	(1.108)	(3.837)
Term on price	$\ln(y - p)$	43.501	44.842	45.898
		(6.427)	(9.216)	(11.748)
Supply-side terms	Constant	0.952	2.760	2.785
		(0.194)	(0.116)	(0.104)
	ln(HP/weight)	0.477	0.897	0.731
		(0.056)	(0.072)	(0.071)
	Air	0.619	0.423	0.528
		(0.038)	(0.087)	(0.040)
	ln(MPG)	−0.415	−0.525	−0.651
		(0.055)	(0.073)	(0.071)
	ln(size)	−0.046	−0.261	−0.472
		(0.081)	(0.210)	(0.125)
	Trend	0.019	0.027	0.018
		(0.002)	(0.003)	(0.002)
Mean own-price elasticity			−3.928	−3.461
Mean markup			0.316	0.346
GMM objective			2.24E-01	1.06E-01
GMM objective scaled by N			4.97E+02	2.36E+02

Note: This table reports replication results for the model of Berry, Levinsohn, and Pakes (1995, 1999) described in Section 6. From left to right, we report estimates from the original article, our replication results, and results using best estimation practices: 10,000 scrambled Halton draws in each market and the "approximate" version of the feasible optimal instruments. Standard errors are in parentheses and are clustered by automobile model.

FIGURE 7

KNITTEL AND METAXOGLOU (2014) HISTOGRAMS FOR MEDIAN PRODUCT OWN-PRICE ELASTICITIES [Color figure can be viewed at wileyonlinelibrary.com]



All figures report the own price elasticity for the median product. The three figures on the left are for the problem in Nevo (2000b). Figures on the right are for a demand-only version of the problem in Berry, Levinsohn, and Pakes (1995) described in Knittel and Metaxoglou (2014). The top two figures are reproduced as fair use from Knittel and Metaxoglou (2014). The bottom four are produced by PyBLP where each observation is one trial from 50 starting values and seven optimization algorithm configurations supported by Knitro and SciPy. The middle figures replicate some of the difficulties in Knittel and Metaxoglou (2014) by using loose optimization tolerances. The bottom two figures eliminate these difficulties with best estimation practices. It suffices to use tight optimization tolerances for the problem in Nevo (2000b). Difficulties for the demand-only version of the problem in Berry, Levinsohn, and Pakes (1995) can be eliminated by additionally using a Gauss-Hermite product rule that exactly integrates polynomials of degree 11 of less instead of 50 pseudo-Monte Carlo draws in each market. For additional details, please consult the online appendix.

- With tighter tolerances and better integration, the dispersion essentially disappears in the replicated examples.
- For the Nevo problem, tight optimization tolerances are enough.
- For the demand-only BLP problem, tight tolerances plus Gauss–Hermite quadrature instead of a small number of pseudo-Monte Carlo draws eliminate the instability.
- The figure is the clearest visual statement of the paper's main practical message: many apparent local-optimum problems are actually numerical-configuration problems.

6 Short summary

- **Method.** The paper organizes modern best practices for estimating BLP-type random-coefficients demand models and implements them in the open-source `PyBLP` package.
- **Core model.** The estimator inverts market shares to mean utilities, stacks demand and supply moments, concentrates out linear parameters, and searches over nonlinear taste and price parameters.
- **Computational recommendations.** Use fixed-effect absorption, SQUAREM or LM for share inversion, analytic gradients, stable log-sum-exp calculations, appropriate integration rules, tight tolerances, and multiple optimizer checks.

- **Instrument recommendation.** Use differentiation or BLP-style instruments for an initial estimate, then construct feasible optimal instruments. When the supply side is credible, combine optimal instruments with supply moments.
- **Main result.** Under these best practices, BLP estimation is substantially more stable than some prior evidence suggested. Local optima are rare in well-identified problems, and finite-sample performance can be good even in small samples.

7 Conclusion

7.1 Core conclusion

- The paper turns BLP estimation from a collection of researcher-specific coding choices into a standardized, inspectable, and extensible workflow.
- It shows that several numerical decisions—fixed-point tolerance, integration rule, optimizer choice, and instrument construction—have first-order consequences for estimates and counterfactuals.
- It also shows that correctly specified supply restrictions and feasible optimal instruments are highly valuable in finite samples.

7.2 Economic intuition

- BLP estimation is hard because the researcher must infer unobserved product quality from market shares while prices are endogenous.
- Instruments identify how price variation changes demand; supply restrictions identify how demand derivatives map into markups and costs.
- Numerical accuracy matters because the model repeatedly solves nonlinear equilibrium objects. Small errors in shares, markups, or integration can propagate into the GMM objective.
- Good instruments and good numerical routines make the objective more informative and easier to optimize.

7.3 Takeaway

This paper is important both as a methodological guide and as an infrastructure contribution. Methodologically, it clarifies how to estimate BLP-type models with supply, demand, optimal instruments, and many fixed effects. Practically, it provides `PyBLP`, which makes those recommendations accessible and replicable. The main lesson is that differentiated-products demand estimation is difficult, but not fragile by necessity: with strong instruments, credible supply restrictions, careful integration, and strict numerical tolerances, BLP models can be estimated reliably and used for serious counterfactual analysis.